

CONNECTIONS BETWEEN SCHEDULE-FREE OPTIMIZERS, AdEMAMix, AND ACCELERATED SGD VARIANTS

Anonymous authors
Anonymous submission

ABSTRACT

Recent developments in deep learning optimization have produced a succession of algorithms—including Schedule-Free optimizers, AdEMAMix, MARS, and Lion—that depart from conventional momentum mechanisms in subtly different ways. In a parallel theoretical literature, acceleration of stochastic gradient descent (SGD) in noise-dominated regimes has been obtained by disentangling the momentum coefficient from the weight assigned to the current gradient. This paper establishes explicit correspondences between these two strands of work. We support the resulting theoretical account with preliminary experiments on a 150m-parameter language-modeling task, where AdEMAMix, the method whose structure most closely mirrors accelerated variants of stochastic gradient descent, attains the strongest performance. Motivated by this alignment, we introduce Simplified-AdEMAMix, a modification that preserves AdEMAMix-level performance in both large- and small-batch settings while obviating the need to maintain two distinct momentum terms. The code for Simplified-AdEMAMix will be released upon acceptance.

1 INTRODUCTION

Recent work in deep learning optimization has introduced a diverse collection of algorithms, including Lion (Chen et al., 2023), ScheduleFreeSGD/AdamW (Defazio et al., 2024), and AdEMAMix (Pagliardini et al., 2024). Although these methods were motivated by different algorithmic desiderata, they are unified by a shared structural intervention: each alters the manner in which momentum is accumulated, weighted, or incorporated into the parameter update.

A distinct theoretical literature has examined the acceleration of gradient-based methods under stochastic, noise-dominated dynamics. While classical momentum mechanisms, including heavy-ball and Nesterov momentum, suffice to accelerate deterministic gradient descent, especially on quadratic objectives, they fail to yield analogous acceleration for SGD (Jain et al., 2018; Liu & Belkin, 2020). This obstruction has motivated alternative momentum constructions designed to preserve acceleration in the presence of gradient noise (Jain et al., 2018; Vaswani et al., 2019; Liu & Belkin, 2020; Gupta et al., 2023). A salient commonality across these accelerated SGD schemes is that they can be understood as decoupling the momentum coefficient from the coefficient assigned to the contemporaneous gradient within the update rule.

Our central contribution is to make explicit the connection between these two lines of inquiry. In particular, we show that Schedule-Free SGD is mathematically equivalent to an accelerated SGD procedure followed by weight averaging. We further argue that optimizers such as Lion, Schedule-Free AdamW, and AdEMAMix may be interpreted as preconditioned variants or extensions of accelerated SGD-style updates. Although particular fragments of this relationship have appeared in prior work (Defazio, 2021), the broader correspondence between these recently proposed optimizers and accelerated SGD variants has, to our knowledge, not previously been formalized.

To assess the empirical relevance of these theoretical observations, we train a 150m-parameter decoder-only transformer on 15b tokens with a 32k-token batch size, thereby placing the optimization trajectory in a noise-dominated regime. Consistent with the proposed equivalence, Schedule-Free AdamW exhibits performance closely aligned with accelerated SGD-based AdamW (Algorithm 3). We also find that accelerated methods yield modest improvements at small batch sizes, whereas these gains attenuate once the batch size becomes sufficiently large, a pattern that accords with the theoretical relationship to accelerated SGD.

The principal contributions of the paper are as follows:

1. We derive precise theoretical connections between accelerated SGD and recently proposed optimizers, including Schedule-Free SGD and AdEMAMix.
2. We supply empirical validation through experiments on a 150-million-parameter decoder-only transformer, comparing AdamW, Schedule-Free AdamW, AdEMAMix, and MARS; in these experiments, AdEMAMix, the optimizer most closely aligned with accelerated SGD variants, achieves the strongest performance.
3. In accordance with the accelerated-SGD equivalence, we show that the advantages of these methods diminish at large batch sizes relative to Adam, and that Adam equipped with momentum scheduling can match the performance of AdEMAMix.
4. At high batch sizes, we observe that Schedule-Free AdamW substantially underperforms AdamW with cosine decay, a degradation that we attribute to the intrinsic coupling between momentum and weight-averaging coefficients in Schedule-Free optimizers.
5. We introduce Simplified-AdEMAMix, a modification that retains AdEMAMix-level performance across both small- and large-batch regimes while eliminating the requirement for two distinct momentum terms.

2 RELATED WORK

We briefly situate our analysis within the literatures on accelerated SGD variants and closely related optimization algorithms.

Jain et al. (2018) introduced an accelerated SGD variant with improved convergence rates for the least-squares problem. Kidambi et al. (2018) subsequently simplified the corresponding update rule and formally established that standard momentum does not provide acceleration in this setting. Later work (Liu & Belkin, 2020; Vaswani et al., 2019; Gupta et al., 2023) extended these acceleration results to broader classes of convex and strongly convex objectives under a range of theoretical assumptions.

Several subsequently proposed optimizers exhibit structural affinities with the accelerated SGD variants described above. Lucas et al. (2019) introduced an update based on a weighted aggregation of multiple momentum terms, each governed by a distinct coefficient, while Ma & Yarats (2019) developed an optimizer explicitly inspired by the theoretical framework of Jain et al. (2018). More recently, Chen et al. (2023) proposed a genetically discovered optimizer that, like accelerated SGD variants, assigns different effective weights to the gradient and to the accumulated momentum in the update. In a similar spirit, Pagliardini et al. (2024) introduced a method whose final update blends two momentum scales.

3 BACKGROUND

3.1 MOMENTUM

Momentum is a canonical mechanism for accelerating gradient descent in deterministic optimization settings. For weights w_t and momentum coefficient β , the standard momentum update is given by:

$$m_t = \beta m_{t-1} + \nabla f(w_t); \quad w_t = w_{t-1} - \eta m_t$$

3.2 WEIGHT AVERAGING

Weight averaging is widely used in stochastic optimization as a variance-reduction device for noisy iterates. Rather than returning only the terminal iterate w_T , one computes a weighted average $\bar{w}_T$ of the optimization trajectory, with averaging weights denoted by γ_t :

$$\bar{w}_T = (1 - \gamma_T)\bar{w}_{T-1} + \gamma_T w_T$$

Throughout this paper, every instance of weight averaging employs coefficients γ_t of the form:

$$\gamma_t \approx 1 - \frac{1}{\delta t}$$

for a constant $0 \leq \delta \leq 1$.

3.3 ACCELERATED SGD

We next introduce a generalized template that subsumes many accelerated SGD methods:

$$m_t = \beta_{a,t} m_{t-1} + g_t, \quad w_{t+1} = w_t - \eta_{a,t} m_t - \alpha_{a,t} g_t \quad (1)$$

where $\beta_{a,t}, \alpha_{a,t}, \eta_{a,t}$ denote possibly time-dependent scalar coefficients, and g_t is the stochastic gradient evaluated at w_t . The subscript ‘a’ is used to distinguish coefficients associated with this accelerated SGD parameterization.

Setting $\alpha_{a,t} = 0$ immediately recovers standard SGD with momentum. Moreover, as prior work implicitly suggests, many accelerated SGD algorithms in the literature—including those of Jain et al. (2018); Vaswani et al. (2019); Liu & Belkin (2020); Gupta et al. (2023)—can be embedded directly within this template; Appendix B gives an explicit derivation of these equivalences.

4 CONNECTIONS BETWEEN EXISTING OPTIMIZERS AND ACCELERATED SGD

This section develops the theoretical correspondences between accelerated SGD and existing optimizers, including Schedule-Free methods and AdEMAMix. Building on these correspondences, we then propose a simplified AdEMAMix variant that uses a single momentum term while maintaining performance comparable to AdEMAMix in both small- and large-batch regimes.

4.1 SCHEDULE-FREE SGD

Schedule-Free SGD (Defazio et al., 2024) is a recently proposed constant-learning-rate optimizer intended to remove the need for an explicit learning-rate schedule. Using the notation of Defazio et al. (2024), its updates are written as:

$$\begin{aligned} y_t &= (1 - \beta) z_t + \beta x_t \\ z_{t+1} &= z_t - \gamma g(y_t) \\ x_{t+1} &= (1 - c_{t+1}) x_t + c_{t+1} z_{t+1} \end{aligned}$$

Here, y_t denotes the current model weights at which the gradient is evaluated, whereas x_t denotes the weights used for evaluation.

We first rewrite the dynamics in terms of y_t and an auxiliary momentum variable m_t , defined by:

$$m_{t+1} = \frac{x_t - z_{t+1}}{\gamma}. \quad (2)$$

Algebraic simplification of this auxiliary quantity yields:

$$m_t = \frac{x_t - z_{t+1}}{\gamma} \quad (3)$$

$$= \frac{x_t + \gamma g_t - z_t}{\gamma} \quad (4)$$

$$= \frac{x_t - z_t + \gamma g_t}{\gamma} \quad (5)$$

$$= \frac{(1 - c_t)(x_{t-1} - z_t) + \gamma g_t}{\gamma} \quad (6)$$

$$= (1 - c_t) m_{t-1} + g_t. \quad (7)$$

Consequently, m_t obeys the momentum recursion in Equation (1) with $\beta_{a,t} = 1 - c_t$. Having identified this momentum variable, we next examine the induced update for y_t :

$$y_{t+1} = (1 - \beta)z_{t+1} + \beta x_{t+1} \quad (8)$$

$$= (1 - \beta)(z_t - \gamma g_t) + \beta((1 - c_{t+1})x_t + c_{t+1}z_{t+1}) \quad (9)$$

$$= (1 - \beta)z_t + \beta x_t - (1 - \beta)\gamma g_t + \beta c_{t+1}(z_{t+1} - x_t) \quad (10)$$

$$= y_t - \gamma[\beta c_{t+1}m_t + (1 - \beta)g_t]. \quad (11)$$

Thus, identifying $w_t = y_t$, the y_t dynamics match the weight update in Equation (1) with $\eta_{a,t} = \gamma\beta c_{t+1}$ and $\alpha_{a,t} = \gamma(1 - \beta)$. The training weights in Schedule-Free SGD therefore instantiate the accelerated SGD framework exactly. Because Schedule-Free SGD nevertheless evaluates using x_t , we must also characterize the dynamics of this evaluation sequence:

$$\begin{aligned} x_{t+1} &= (1 - c_{t+1})x_t + c_{t+1}z_{t+1} \\ x_{t+1} &= (1 - c_{t+1})x_t + c_2 \left(\frac{y_{t+1} - \beta x_{t+1}}{1 - \beta} \right) \\ x_{t+1}(1 - \beta + c_{t+1}\beta) &= (1 - c_{t+1})(1 - \beta)x_t + c_{t+1}y_{t+1} \\ x_{t+1} &= \frac{(1 - c_{t+1})(1 - \beta)x_t + c_{t+1}y_{t+1}}{(1 - c_{t+1})(1 - \beta) + c_{t+1}}. \end{aligned}$$

It follows that x_t is a weighted average of the y_t iterates. Recursive expansion shows, in particular, that when c_t is constant, x_t becomes an exponential average of y_t . Schedule-Free SGD can therefore be interpreted as accelerated SGD followed by weight averaging.

Under this interpretation, the advantages of Schedule-Free SGD decompose into two components:

1. Improved performance relative to standard momentum SGD, arising from its equivalence to accelerated SGD.
2. The ability to operate with a constant learning rate, without explicit scheduling, through weight averaging—more precisely, tailed weight averaging; see Section 4.1.3.

Schedule-Free SGD/Adam also retains two implementation-level advantages:

- It avoids additional memory allocation for weight averaging.
- It removes the explicit weight-averaging coefficient from the hyperparameter set.

However, Section 5.1 shows that the induced coupling between momentum and weight-averaging coefficients scales poorly in large-batch settings.

4.1.1 CASE: $\beta = 0.0$

As noted in (Defazio et al., 2024), setting $\beta = 0$ reduces Schedule-Free SGD to standard SGD with weight averaging; because $c_t = 1/t$, averaging is applied from the initial iterations onward.

4.1.2 CASE: $\beta = 1.0$

As noted in (Defazio et al., 2024), setting $\beta = 1$ yields standard momentum SGD, with the momentum coefficient $\beta_{a,t}$ scaling as $1 - 1/t$.

4.1.3 CASE: $\beta = 0.9$

For the default Schedule-Free SGD setting $\beta = 0.9$, the induced dynamics have the following consequences:

- Because c_t scales as $1/t$, the effective momentum coefficient grows as $1 - 1/t$.

- The ratio between the current-gradient weight and the momentum weight is fixed at $(1 - \beta)/(\beta c_{t+1}) \approx 0.11$.
- Weight averaging is effectively concentrated on approximately the most recent 10% of iterates.

4.2 LION

The Lion (Chen et al., 2023) update can be written as:

$$m'_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$\theta_t = \theta_{t-1} - \eta \text{sign}(m'_t)$$

$$m_t = \beta_2 m_{t-1} + (1 - \beta_2) g_t.$$

Lion (Chen et al., 2023) therefore admits a direct interpretation as an accelerated SGD-style update followed by a coordinate-wise sign transformation.

4.3 MARS

We next show that the practical version of the recently proposed MARS optimizer (Yuan et al., 2024), referred to as MARS-Approx, conforms to the accelerated SGD template once augmented with a preconditioning step. Omitting bias correction and clipping, its updates are:

$$\begin{aligned} c_t &= g_t + \gamma \frac{\beta_1}{1 - \beta_1} [g_t - g_{t-1}] \\ m_t &= \beta_1 m_{t-1} + (1 - \beta_1) c_t \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) c_t^2 \\ x_{t+1} &= x_t - \eta \frac{m_t}{\sqrt{v_t} + \epsilon} \end{aligned}$$

where m_t and v_t denote the first- and second-order momentum quantities, respectively, and x_t denotes the model parameters. Reparameterizing the update via $\hat{m}_t = m_t - \gamma g_t$ gives:

$$\begin{aligned} c_t &= g_t + \gamma \frac{\beta_1}{1 - \beta_1} [g_t - g_{t-1}] \\ \hat{m}_t &= \beta_1 \hat{m}_{t-1} + (1 - \beta_1)(1 - \gamma) g_t \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) c_t^2 \\ x_{t+1} &= x_t - \eta \frac{\hat{m}_t + \gamma g_t}{\sqrt{v_t} + \epsilon} \end{aligned}$$

This form makes clear that the momentum recursion follows the general accelerated SGD framework. Nevertheless, MARS differs from AdamW in its preconditioning construction, and Section 5 examines the empirical consequences of this distinction.

4.4 ADEMAMIX

The recently proposed AdEMAMix optimizer (Pagliardini et al., 2024) is structurally close to accelerated SGD-based AdamW. Rather than forming, as accelerated SGD does, a linear combination of the current gradient and a single momentum term, AdEMAMix maintains two momentum estimates with distinct coefficients and combines them linearly in the final update. Algorithm 1 states the algorithm formally.

For analytical clarity, we consider the AdEMAMix variant with $\beta_1 = 0$. As shown by Pagliardini et al. (2024), this reduced version performs nearly as well as the full optimizer at small batch sizes, and our experiments in Section 5 corroborate that observation. Under $\beta_1 = 0$, AdEMAMix aligns with the general accelerated SGD framework (Equation (1)); moreover, its prescribed schedules for β_3 , the momentum coefficient, and α , which controls the relative contribution of the current gradient, closely resemble theoretical schedules proposed for accelerated SGD (Gupta et al., 2023).

In smooth convex optimization, acceleration under stochastic gradients requires a momentum schedule of the form:

$$\beta_{a,t} = 1 - \frac{k}{t}$$

for some constant $k > 0$, as established by Gupta et al. (2023). AdEMAMix approximately realizes this prescription through the corresponding increase of β_3 .

It is also useful to observe that accelerated SGD schemes maintain momentum in the unnormalized form:

$$m_t = \beta_{a,t} m_{t-1} + g_t$$

whereas the AdEMAMix update in Algorithm 1 uses the normalized EMA form:

$$m_2^{(t)} \leftarrow \beta_3^{(t)} m_2^{(t-1)} + (1 - \beta_3^{(t)}) g^{(t)}.$$

When $\beta_{a,t}$ scales as $1 - 1/t$, the aggregate contribution of historical gradients in the accelerated-SGD momentum variable m_t grows proportionally to t . The coefficient α in AdEMAMix is scaled in an analogous manner. These structural similarities provide a plausible explanation for the improved empirical performance of AdEMAMix relative to the other optimizers in Figure 1.

At large batch sizes, however, AdEMAMix suffers a performance decline when $\beta_1 = 0.0$, as reported by Pagliardini et al. (2024). The analysis of Gupta et al. (2023) suggests that, in such regimes, the coefficient on the current gradient should decrease. AdEMAMix instead retains a fixed unit weight on the current gradient, a design choice that likely contributes to its reduced large-batch performance.

The next subsection introduces a simplified AdEMAMix variant that explicitly weights the current gradient, eliminates the need to schedule α , and maintains only a single momentum term. We empirically verify that this simplification remains competitive with AdEMAMix in both small- and large-batch experiments.

Algorithm 1 Single step of AdEMAMix optimizer.

- 1: **Input:** Data distribution $\mathcal{D}$. Initial model parameters $\theta^{(0)}$. Number of iterations T . Learning rate η . ϵ a small constant. AdamW parameters: β_1, β_2 . AdEMAMix parameters β_3, α . Warmup parameter T_{α, β_3} , note that we usually set it to T . β_{start} is usually set to β_1 .
 - 2: Optional: use schedulers $\eta^{(t)}, \beta_3^{(t)} \leftarrow f_{\beta_3}(t, \beta_3, \beta_{\text{start}}, T_{\alpha, \beta_3})$ and $\alpha^{(t)} \leftarrow f_{\alpha}(t, \alpha, T_{\alpha, \beta_3})$
 - 3: Sample batch: $x \sim \mathcal{D}$
 - 4: Compute gradient: $g^{(t)} \leftarrow \nabla_{\theta} \mathcal{L}_{\theta^{(t-1)}}(x)$
 - 5: Update the fast EMA m_1 : $m_1^{(t)} \leftarrow \beta_1 m_1^{(t-1)} + (1 - \beta_1) g^{(t)}$
 - 6: Update the slow EMA m_2 : $m_2^{(t)} \leftarrow \beta_3^{(t)} m_2^{(t-1)} + (1 - \beta_3^{(t)}) g^{(t)}$
 - 7: Update the second moment estimate: $\nu^{(t)} \leftarrow \beta_2 \nu^{(t-1)} + (1 - \beta_2) (g^{(t)})^2$
 - 8: Update parameters: $\theta^{(t)} \leftarrow \theta^{(t-1)} - \eta^{(t)} \left(\frac{\hat{m}_1^{(t)} + \alpha^{(t)} m_2^{(t)}}{\sqrt{\hat{\nu}^{(t)} + \epsilon}} \right)$
-

4.5 SIMPLIFIED ADEMAMIX

Building on the preceding analysis, we propose a simplified optimizer that dispenses with two separate momentum terms and removes the requirement to schedule α . The method, presented formally in Algorithm 2, uses theory-style momentum rather than an exponential-moving-average (EMA) parameterization. Its final update assigns a fixed

Algorithm 2 Single step of Simplified AdEMAMix optimizer.

- 1: **Input:** Data distribution $\mathcal{D}$. Initial model parameters $\theta^{(0)}$. Number of iterations T . Learning rate η . ϵ a small constant. AdamW parameters: β_1, β_2 . AdEMAMix parameters $\alpha, \beta_{\text{start}}$. Warmup parameter T_{β_1} , note that we usually set it to T .
 - 2: Optional: use schedulers $\eta^{(t)}, \beta_1^{(t)} \leftarrow f_{\beta_1}(t, \beta_1, \beta_{\text{start}}, T_{\beta_1})$
 - 3: Sample batch: $x \sim \mathcal{D}$
 - 4: Compute gradient: $g^{(t)} \leftarrow \nabla_{\theta} \mathcal{L}_{\theta^{(t-1)}}(x)$
 - 5: Update the EMA m_1 : $m_1^{(t)} \leftarrow \beta_1 m_1^{(t-1)} + g^{(t)}$
 - 6: Update the second moment estimate: $\nu^{(t)} \leftarrow \beta_2 \nu^{(t-1)} + (1 - \beta_2) (g^{(t)})^2$
 - 7: Update parameters: $\theta^{(t)} \leftarrow \theta^{(t-1)} - \eta^{(t)} \left(\frac{m_1^{(t)} + \alpha g^{(t)}}{\sqrt{\hat{\nu}^{(t)}} + \epsilon} \right)$
-

coefficient α to the current gradient; setting $\alpha = 0$ recovers standard Adam after the appropriate transformations of η and β_1 . In Section 5, we show that this simplified variant matches AdEMAMix across both small and large batch sizes.

5 EXPERIMENTS

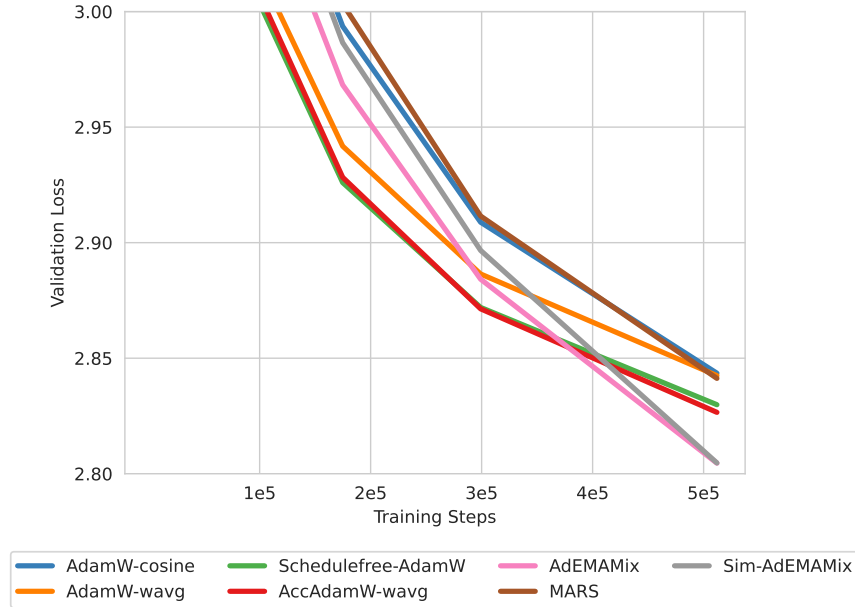


Figure 1: Comparison of the best runs for the optimizers evaluated in Section 5 on a language-modeling task with a decoder-only 150m-parameter transformer. AdEMAMix and simplified-AdEMAMix achieve the strongest performance, consistent with their close structural resemblance to accelerated SGD variants.

We evaluate the preceding theoretical claims on a language-modeling task using the C4 dataset and a 150-million-parameter decoder-only transformer. The model is trained with sequence length 1024 and batch size 32 for 15 billion tokens ($\approx 5 \times$ Chinchilla), a configuration chosen to place training in a noise-dominated regime.

The comparison includes the following optimization algorithms:

1. Standard AdamW with cosine decay
2. Standard AdamW with weight averaging

Algorithm 3 Single step of accelerated SGD based Adam with weight averaging. For simplicity we ignore the initialization, other boundary effects such as bias correction, and weight decay. Hyperparameters: Learning rate η , betas = $(\beta_1, \beta_2, \beta_3)$, weight averaging coefficient δ , and epsilon ϵ .

```

1: Sample batch  $B_t$ .
2:  $g \leftarrow -\nabla_w \phi_{B_t}(w_t)$ 
3:  $v \leftarrow \beta_2 v + (1 - \beta_2)(g \odot g)$ 
4:  $N \leftarrow \frac{\beta_3 m + (1 - \beta_3)g}{\sqrt{v} + \epsilon}$ 
5:  $w \leftarrow w - \eta N$ 
6:  $m \leftarrow \beta_1 m + (1 - \beta_1)g$ 
7:  $c = \max(1 - 1/t, 1 - 1/(\delta t))$ 
8:  $w_{\text{avg}} \leftarrow c w_{\text{avg}} + (1 - c)w$ 

```

3. Schedule-Free AdamW
4. Accelerated AdamW with weight averaging (Algorithm 3)
5. MARS
6. AdEMAMix
7. Simplified-AdEMAMix

The hyperparameter sweeps associated with these algorithms are reported in Section A.

As shown in Figure 1, Schedule-Free AdamW and Accelerated AdamW with tailed weight averaging attain comparable performance, thereby supporting the theoretical equivalence developed above. Both methods outperform AdamW with cosine decay and AdamW with tailed weight averaging. AdEMAMix and Simplified-AdEMAMix further outperform all evaluated methods, a result we hypothesize follows from their closer alignment with accelerated SGD variants.

5.1 LARGE BATCH SIZE EXPERIMENTS

Whereas the preceding experiments target the small-batch regime, in which gradient noise is substantial, we next examine whether the same algorithmic behavior persists at large batch sizes. In this setting, models are trained with a 1-million-token batch size for 3 billion tokens ($\approx$ Chinchilla scale).

Schedule-Free AdamW: As shown in Figure 2, Schedule-Free AdamW substantially underperforms AdamW. We attribute this gap to the coupling between the weight-averaging and momentum coefficients: at larger batch sizes, the optimal momentum value is considerably smaller than $1 - 1/t$. Although a scaling factor of the form $\approx 1 - r/t$, for some $r \geq 1$, could be used, increasing r correspondingly shortens the effective weight-averaging window.

A second distinction between AdamW and Schedule-Free AdamW concerns the ordering of momentum and preconditioning. AdamW applies momentum prior to preconditioning, whereas Schedule-Free AdamW applies preconditioning before momentum, rendering it algorithmically similar to LProp (Ziyin et al., 2021). Nevertheless, Figure 2 shows that AdamW and LProp perform comparably, suggesting that this ordering difference is not the principal source of the observed performance gap.

AdEMAMix: In the large-batch regime, consistent with the observations of Pagliardini et al. (2024), Figure 3 shows that setting $\beta_1 = 0.0$ in AdEMAMix causes a substantial degradation relative to using two separate momentum terms. This degradation arises because AdEMAMix assigns a fixed unit weight to the current gradient, whereas theoretical accelerated SGD variants (Gupta et al., 2023) require the current-gradient weight to diminish as the batch size increases.

In addition, Figure 3 shows that Simplified-AdEMAMix matches the performance of AdEMAMix while eliminating the need for two distinct momentum terms. Notably, this performance is obtained at $\alpha = 0.0$, in which case Simplified-AdEMAMix reduces to standard Adam with momentum scheduling.

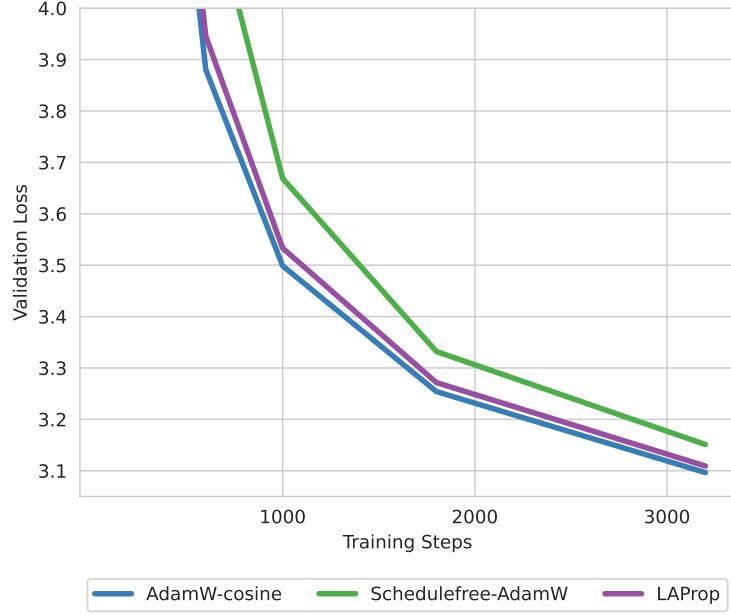


Figure 2: Comparison of the best high-batch-size runs for AdamW with cosine decay, schedule free AdamW, and LARProp. Experimental details are provided in Section 5.1.

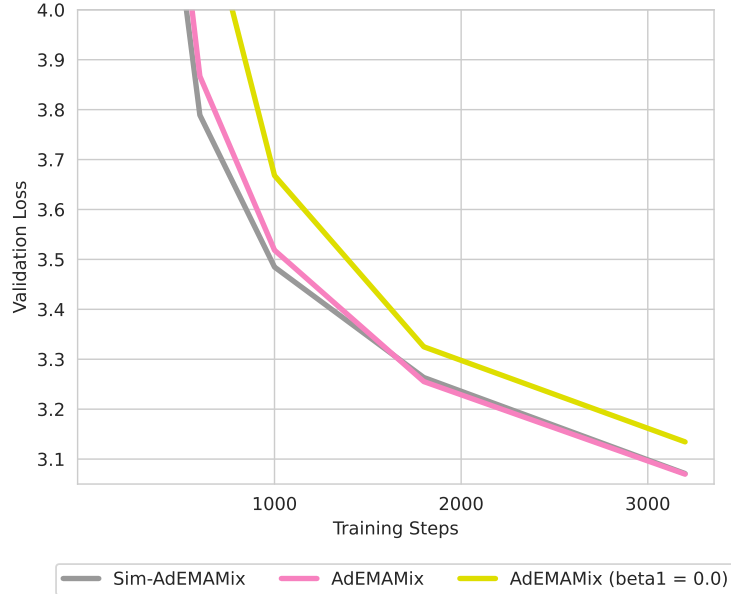


Figure 3: Comparison of the best high-batch-size runs for AdEMAMix, with and without $\beta_1 = 0.0$, and for our simplified AdEMAMix variant. Experimental details are provided in Section 5.1.

6 CONCLUSION

This work establishes explicit connections between accelerated SGD variants and several recently proposed optimizers, including Schedule-Free optimizers, AdEMAMix, MARS, and Lion. The accompanying experiments show that

AdEMAMix, whose update structure most closely aligns with theoretical accelerated SGD variants, achieves superior performance in small-batch training.

Motivated by this connection, we introduce Simplified-AdEMAMix, which removes the need to maintain two separate momentum buffers. Empirically, Simplified-AdEMAMix matches AdEMAMix across both small and large batch sizes while eliminating the additional memory overhead associated with the original AdEMAMix formulation.

REFERENCES

- Chen, X., Liang, C., Huang, D., Real, E., Wang, K., Pham, H., Dong, X., Luong, T., Hsieh, C., Lu, Y., and Le, Q. V. Symbolic discovery of optimization algorithms. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S. (eds.), *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023. URL http://papers.nips.cc/paper_files/paper/2023/hash/9a39b4925e35cf447ccba8757137d84f-Abstract-Conference.html.
- Defazio, A. Momentum via primal averaging: Theoretical insights and learning rate schedules for non-convex optimization, 2021. URL <https://arxiv.org/abs/2010.00406>.
- Defazio, A., Yang, X., Mehta, H., Mishchenko, K., Khaled, A., and Cutkosky, A. The road less scheduled. *CoRR*, abs/2405.15682, 2024. doi: 10.48550/ARXIV.2405.15682. URL <https://doi.org/10.48550/arXiv.2405.15682>.
- Gupta, K., Siegel, J., and Wojtowysch, S. Achieving acceleration despite very noisy gradients, 2023. URL <https://arxiv.org/abs/2302.05515>.
- Jain, P., Kakade, S. M., Kidambi, R., Netrapalli, P., and Sidford, A. Accelerating stochastic gradient descent for least squares regression. In Bubeck, S., Perchet, V., and Rigollet, P. (eds.), *Proceedings of the 31st Conference On Learning Theory*, volume 75 of *Proceedings of Machine Learning Research*, pp. 545–604. PMLR, 06–09 Jul 2018. URL <https://proceedings.mlr.press/v75/jain18a.html>.
- Kidambi, R., Netrapalli, P., Jain, P., and Kakade, S. M. On the insufficiency of existing momentum schemes for stochastic optimization. In *International Conference on Learning Representations*, 2018. URL <https://openreview.net/forum?id=rJTutzbA->.
- Liu, C. and Belkin, M. Accelerating sgd with momentum for over-parameterized learning. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=rlgixp4FPH>.
- Lucas, J., Sun, S., Zemel, R., and Grosse, R. Aggregated momentum: Stability through passive damping. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=Syxt5oC5YQ>.
- Ma, J. and Yarats, D. Quasi-hyperbolic momentum and adam for deep learning. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=SlfUpOR5FQ>.
- Pagliardini, M., Ablin, P., and Grangier, D. The ademamix optimizer: Better, faster, older. 2024. URL <https://arxiv.org/abs/2409.03137>.
- Porian, T., Wortsman, M., Jitsev, J., Schmidt, L., and Carmon, Y. Resolving discrepancies in compute-optimal scaling of language models. *CoRR*, abs/2406.19146, 2024. doi: 10.48550/ARXIV.2406.19146. URL <https://doi.org/10.48550/arXiv.2406.19146>.
- Vaswani, S., Bach, F., and Schmidt, M. Fast and faster convergence of sgd for over-parameterized models and an accelerated perceptron. In Chaudhuri, K. and Sugiyama, M. (eds.), *Proceedings of the Twenty-Second International Conference on Artificial Intelligence and Statistics*, volume 89 of *Proceedings of Machine Learning Research*, pp. 1195–1204. PMLR, 16–18 Apr 2019. URL <https://proceedings.mlr.press/v89/vaswani19a.html>.
- Yuan, H., Liu, Y., Wu, S., Zhou, X., and Gu, Q. Mars: Unleashing the power of variance reduction for training large models, 2024. URL <https://arxiv.org/abs/2411.10438>.

Ziyin, L., Wang, Z. T., and Ueda, M. Laprop: Separating momentum and adaptivity in adam, 2021. URL <https://arxiv.org/abs/2002.04839>.

A HYPERPARAMETERS

For reproducibility, the hyperparameter configurations considered in the small-batch experiments are enumerated below, with each item specifying the grid over which the corresponding optimizer was tuned.

1. AdamW with cosine decay - 51.2k warmup - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.9, 0.95]$, β_2 in $[0.99, 0.999, 0.99968, 0.9999]$. The optimal values of β_1 and β_2 were .9 and .999, respectively, thereby coinciding with the default settings. Although larger-batch training commonly uses $\beta_2 = .95$, the advantage of a higher β_2 value at smaller batch sizes has also been reported by Porian et al. (2024).
2. AdamW with cosine decay - 10k warmup - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ i.e. we fix β_1, β_2 to be the optimal values from the previous sweep. This shorter-warmup configuration performed worse than the 51.2k-step warmup.
3. AdamW constant fraction weight averaging: - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, $\beta_1 = 0.9$, β_2 in $[0.99, 0.997, 0.999, 0.9997]$, δ in $[0.05, 0.1, 0.2]$.
4. AdamW with cosine decay and weight averaging - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, δ in $[0.025, 0.05, 0.1]$.
5. Accelerated SGD based AdamW with cosine decay - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.999, 0.99968, 0.9999]$, β_2 in $[0.99, 0.9968, 0.999]$, $\beta_3 = 0.9$
6. Accelerated SGD based AdamW with constant learning rate and weight averaging - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.99684, 0.999]$, β_2 in $[0.999]$, $\beta_3 = 0.9$, δ in $[0.05, 0.1]$
7. Accelerated SGD based AdamW with cosine decay and weight average - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.99684, 0.999]$, $\beta_2 = 0.999$, δ in $[0.05, 0.1]$, $\beta_3 = 0.9$
8. Schedulefree AdamW with constant learning rate - learning rate in $[3.16e-4, 1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.8, 0.9, 0.95]$, $\beta_2 = 0.999$
9. Schedulefree AdamW with cosine decay - $[3.16e-4, 1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.8, 0.9, 0.95]$, $\beta_2 = 0.999$
10. MARS - $[3.16e-4, 1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.9, 0.95, 0.99]$, β_2 in $[0.99, 0.999]$, γ in $[0.0, 0.01, 0.02, 0.03, 0.04, 0.05]$, precondition 1d was set to True.
11. AdEMAMix - $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.0, 0.9]$, $\beta_2 = 0.999$, β_3 in $[0.99, 0.999, 0.9999]$, α in $[2, 4, 8, 16]$.
12. Sim-AdEMAMix - $[1e-6, 3.16e-6, 1e-5, 3.16e-5]$, β_1 in $[0.99, 0.999, 0.9999]$, $\beta_2 = 0.999$, α in $[10, 20, 50, 100]$

The analogous hyperparameter sweeps for the large-batch experiments are specified as follows, preserving the same optimizer-specific organization while adapting the candidate ranges to the altered batch-size regime:

1. Schedule-Free AdamW: $[1e-3, 3.16e-3, 1e-2]$, β in $[0.8, 0.9, 0.95]$, β_2 in $[0.9, 0.95]$, r in $[0.0, 5.0, 9.0, 50.0]$
2. AdamW: $[1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.9, 0.95]$, β_2 in $[0.9, 0.95]$
3. LProp: $[1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.9, 0.95]$, β_2 in $[0.9, 0.95]$
4. AdEMAMix: $[1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.0, 0.9]$, $\beta_2 = 0.95$, β_3 in $[0.9, 0.95, 0.99]$, α in $[2, 4, 8, 16]$
5. Sim-AdEMAMix: $[1e-4, 3.16e-4, 1e-3]$, β_1 in $[0.9, 0.95, 0.99]$, $\beta_2 = 0.95$, α in $[0.0, 0.5, 1.0]$

B EQUIVALENCE OF PREVIOUS ACCELERATION METHODS

Equation (1) presents the general accelerated SGD template. This appendix verifies, through direct algebraic rearrangement, that the methods of Jain et al. (2018); Vaswani et al. (2019); Liu & Belkin (2020); Gupta et al. (2023) can all be expressed within that common template.

B.1 AGNES

The update rule introduced by Gupta et al. (2023) can be written as follows:

$$x'_n = x_n + \alpha v_n \quad x_{n+1} = x'_n - \eta g'_n \quad v_{n+1} = \rho_n(v_n - g'_n)$$

where g'_n denotes the stochastic gradient evaluated at x'_n , whereas the final function value is evaluated at x_n . By collecting the velocity-dependent terms and rescaling the momentum recurrence, these equations may be rewritten as

$$x'_{n+1} = x'_n - \eta g'_n + \alpha v_{n+1} \quad -\frac{v_{n+1}}{\rho_n} = \rho_{n-1} \left(-\frac{v_n}{\rho_{n-1}} \right) + g'_n$$

Thus x'_{n+1} satisfies an update of the form given in Equation (1).

B.2 ASGD

The update rule of Jain et al. (2018) is given by:

$$y_{j-1} = \alpha x_{j-1} + (1 - \alpha)v_{j-1} \quad x_j = y_{j-1} - \delta g_{j-1} \quad z_{j-1} = \beta y_{j-1} + (1 - \beta)v_{j-1} \quad v_j = z_{j-1} - \gamma g_{j-1}$$

where g_{j-1} denotes the stochastic gradient evaluated at y_{j-1} , while the function value is evaluated on the tail-averaged iterate x .

After eliminating the auxiliary variables in favor of the displacement between y and v , these update equations can be rearranged as:

$$y_j = y_{j-1} - \alpha \delta g_{j-1} - (1 - \alpha)[y_{j-1} - v_j] \quad \frac{y_{j-1} - v_j}{\gamma - (1 - \beta)\alpha\delta} = (1 - \beta)\alpha \frac{y_{j-2} - v_{j-1}}{\gamma - (1 - \beta)\alpha\delta} + g_{j-1}$$

Consequently, the resulting recurrence has precisely the form of Equation (1).

B.3 MASS

The MaSS update of Liu & Belkin (2020) is:

$$w_{t+1} = u_t - \eta_1 g_t \quad u_{t+1} = (1 + \gamma)w_{t+1} - \gamma w_t + \eta_2 g_t$$

where g_t is the stochastic gradient evaluated at u_t , whereas the function is evaluated at w_t . Reexpressing the dynamics in terms of the discrepancy $w_t - u_t$ yields:

$$u_{t+1} = u_t - \gamma(w_t - u_t) - [\eta_1(1 + \gamma) - \eta_2]g_t \quad \frac{w_t - u_t}{\eta_1\gamma - \eta_2} = \gamma \frac{w_{t-1} - u_{t-1}}{\eta_1\gamma - \eta_2} + g_t$$

Consequently, the update sequence again instantiates the template in Equation (1).

B.4 SGD WITH NESTEROV ACCELERATION

The Nesterov-accelerated SGD update analyzed by Vaswani et al. (2019) is:

$$w_{k+1} = \zeta_k - \eta g_k \quad \zeta_k = \alpha_k v_k + (1 - \alpha_k)w_k \quad v_{k+1} = \beta_k v_k + (1 - \beta_k)\zeta_k - \gamma_k \eta g_k$$

These equations admit the following rearrangement, which isolates the accelerated displacement term and the corresponding gradient contribution:

$$\zeta_{k+1} = \zeta_k - \eta g_k + \alpha_{k+1}[v_{k+1} - w_{k+1}]; \quad v_{k+1} - w_{k+1} = \beta_k(1 - \alpha_k)[v_k - w_k] - \eta(\gamma_k - 1)g_k$$